Cuadernillo de simulación de espines — manual de uso

Esta carpeta contiene todo lo necesario para simular un material magnético con el cuadernillo `spin_simulation.ipynb`. No necesita saber programar ni instalar el repositorio: el cuadernillo lleva su propio código.

En una frase: abra el cuadernillo en Kaggle o en Google Colab, cambie los números de **una sola celda**, pulse **Run all** (ejecutar todo) y descargue los resultados.

1. Qué hace el cuadernillo

Simula espines (pequeños imanes) ubicados en los sitios de un cristal. Para cada temperatura, de caliente a frío, deja que los espines lleguen al equilibrio y luego mide:

- la **energía** E,
- la **magnetización** M (qué tan alineados están los espines),
- el **calor específico** Cv y la **susceptibilidad** χ (sus picos marcan la transición de fase),
- la **configuración** completa: la dirección de cada espín en cada copia del sistema.

Puede simular varios **estados** a la vez. Un estado es un conjunto de parámetros del hamiltoniano, por ejemplo ”sin campo” y ”con campo”. Cada estado corre muchas copias independientes del sistema, llamadas **réplicas**, y los resultados se promedian sobre ellas.

2. Dónde ejecutarlo

Dónde	Qué necesita	Velocidad
Kaggle (recomendado)	Una cuenta gratuita verificada por teléfono (necesario para las GPU)	Rápido: dos GPU T4
Google Colab	Una cuenta de Google	Rápido: una GPU T4
Su propio computador	Linux (o Windows con WSL) y una GPU NVIDIA	Depende de su GPU
Cualquiera de ellos sin GPU	Nada más	**Muy lento**: solo para pruebas pequeñas

El cuadernillo detecta el hardware por sí solo: usa todas las GPU NVIDIA que encuentre, y la CPU si no hay ninguna. La GPU debe tener capacidad de cómputo 7.0 o mayor (T4, V100, A100, H100, RTX 20xx y posteriores). Las GPU más antiguas, como la P100, no son compatibles; en ese caso el cuadernillo usa la CPU.

2.1 Kaggle, paso a paso

1. Entre a `https://www.kaggle.com`, inicie sesión y haga clic en **Create → New Notebook**.

2. En el menú elija **File → Import Notebook** y suba `spin_simulation.ipynb`.
3. En el panel derecho abra **Session options** y ponga **Accelerator** en **GPU T4 x2**.
4. Edite la celda 1 (vea la sección 3).
5. Haga clic en **Run all**.
6. Al terminar, los resultados están en la sección **Output** del panel derecho, en la carpeta `simulation_output`. Descárguelos desde ahí.

Una sesión de Kaggle dura como máximo 12 horas, y la cuota gratuita de GPU es de unas 30 horas por semana.

2.2 Google Colab, paso a paso

1. Entre a `https://colab.research.google.com`, elija **Subir** (Upload) y seleccione `spin_simulation.`
2. Menú **Entorno de ejecución → Cambiar tipo de entorno de ejecución → GPU T4 → Guardar**.
3. Edite la celda 1.
4. Menú **Entorno de ejecución → Ejecutar todas**.
5. Para descargar los resultados en un solo archivo, agregue una celda nueva al final con la línea de abajo, ejecútela y descargue `simulation_output.zip` desde el ícono de carpeta de la izquierda:

```
!zip -r simulation_output.zip simulation_output
```

2.3 Su propio computador

```
pip install torch triton numpy matplotlib jupyter
jupyter notebook spin_simulation.ipynb
```

Luego edite la celda 1 y elija **Run → Run All Cells**. Los resultados quedan junto al cuadernillo.

3. La única celda que se edita: la celda 1

La celda 1 está dividida en ocho bloques. Cada línea tiene al lado un comentario corto (en inglés). Cambie solo los valores que están después del signo `=`, y conserve las comillas, los corchetes y las comas.

3.1 Tipo de hamiltoniano — `MODEL`

Valor	Espines	Ejemplo
`"ising"`	arriba o abajo (±1)	`MODEL = "ising"`
`"xy"`	flechas que giran en un plano	`MODEL = "xy"`
`"heisenberg"`	flechas que giran en 3D	`MODEL = "heisenberg"`

3.2 El hamiltoniano

La energía que usa el cuadernillo es la suma de cuatro términos:

- **Intercambio** `J`: los espines vecinos quieren estar paralelos (J > 0, ferromagneto) u opuestos (J < 0, antiferromagneto). Se da un valor por cada **capa** de vecinos: capa 1 = primeros vecinos, capa 2 = segundos vecinos, y así sucesivamente.
- **DMI** `D`: interacción de Dzyaloshinskii–Moriya de tipo bulk (Bloch): su vector apunta a lo largo de cada enlace. Retuerce los espines y crea skyrmiones de tipo Bloch. Actúa sobre los enlaces de primeros vecinos que están en el plano x–y.
- **Anisotropía cúbica** `K1`, `Ks`, `K2`: favorece algunas direcciones del cristal. `Ks` es el valor en la **superficie** (sitios con menos vecinos que en el interior); si no lo escribe, vale lo mismo que `K1`.
- **Campo** `H`: un campo magnético externo. Su dirección se da con dos ángulos en grados: `H_theta` (0 = a lo largo de +z, la normal de una película o un disco; 90 = en el plano) y `H_phi` (el ángulo en el plano x–y, 0 = a lo largo de +x).

`D`, `K1`, `Ks` y `K2` solo existen para `"heisenberg"`. Ising y XY usan solo `J` y `H`.

3.3 Los estados — `STATES`

Cada estado es una línea `dict(...)` dentro de los corchetes, terminada en coma. Escriba solo los parámetros que necesite: **lo que no escriba vale cero**.

```
STATES = [
    dict(name="no field",   J={1: 1.0}),
    dict(name="with field", J={1: 1.0}, H=0.2, H_theta=0.0),
    dict(name="frustrated", J={1: 1.0, 2: -0.3}),
]
```

- `name` es la etiqueta que aparece en las gráficas; cada estado necesita un nombre distinto.
- `J={1: 1.0, 2: -0.3}` significa J = 1 para primeros vecinos y J = −0.3 para la segunda capa.
- Para desactivar un estado sin borrarlo, ponga `#` al comienzo de su línea.

3.4 Unidades — `UNITS`

Valor	Energías	Temperatura
`"reduced"`	en unidades de su J (J = 1)	en las mismas unidades, k_B = 1
`"meV"`	en meV	en kelvin

3.5 Cristal y caja — `CRYSTAL`, `BOX`, `PERIODIC`

Dimensión	Cristales	Caja
2D	`"square"`, `"triangular"`, `"honeycomb"`, `"kagome"`	`BOX = [Lx, Ly]`
3D	`"sc"`, `"bcc"`, `"fcc"`	`BOX = [Lx, Ly, Lz]`

- La caja se cuenta en **celdas unitarias**. Tamaños iguales dan una caja cuadrada (o cúbica); tamaños distintos dan una caja rectangular. bcc tiene 2 sitios por celda y fcc tiene 4.
- `PERIODIC` lleva un `True` o `False` por cada dirección. `True` significa que la caja se repite (sin bordes), como en un material masivo; `False` significa que la caja tiene bordes.
- Algunas cajas periódicas no se pueden colorear con el método; el cuadernillo lo avisa y usa bordes abiertos.

3.6 Sitios activos — `COORDS` y `SHAPE`

Por defecto todos los sitios de la caja llevan un espín. Para simular una **figura** (una esfera, un disco, un hilo...), primero elija un sistema de coordenadas y luego escriba la figura en él. El origen es el **centro de la caja**, las longitudes están en unidades de red y los ángulos en radianes.

`COORDS`	Nombres que puede usar	Ejemplo de `SHAPE`
`"cartesian"`	`x`, `y`, `z`	`"(abs(x) <= 5) & (abs(y) <= 3)"` (un rectángulo)
`"spherical"`	`r`, `theta`, `phi`	`"r <= 8"` (una esfera de radio 8)
`"cylindrical"`	`rho`, `phi`, `z`	`"(rho <= 10) & (abs(z) <= 2)"` (un disco)

- Una las condiciones con `&` (y), `|` (o) y `~` (no), y ponga cada condición entre paréntesis. Un anillo: `"(rho <= 10) & (rho >= 6)"`.
- **Para dejar todos los sitios activos, deje ambos vacíos:** `COORDS = ""` y `SHAPE = ""`.
- Use una caja suficientemente grande para contener la figura, y `PERIODIC = [False, False, False]` cuando use una figura.

3.7 Temperaturas y barridos

Parámetro	Significado	Valor típico
`T_MAX`	primera temperatura (la más caliente)	por encima de la transición
`T_MIN`	última temperatura (la más fría)	por debajo de la transición
`T_STEP`	distancia entre dos temperaturas	0.05 a 0.1
`THERMALIZATION`	barridos para llegar al equilibrio en cada temperatura	1000 a 10000
`MEASUREMENT`	barridos que se miden en cada temperatura	2000 a 20000
`MEASURE_EVERY`	medir una vez cada tantos barridos	10
`REPLICAS`	copias independientes del sistema por estado	32 a 128
`SEED`	semilla de los números aleatorios; la misma semilla repite la misma corrida	cualquier entero

Un **barrido** significa que cada espín tiene una oportunidad de cambiar. Más barridos dan curvas más suaves y tardan más. En una GPU, más réplicas cuestan poco tiempo adicional por espín, así que 64 o 128 es una buena elección.

3.8 Salida — `CONFIG_EVERY`, `OUTPUT_DIR`, `GPUS`

- `CONFIG_EVERY = 1` guarda la configuración de cada réplica en **cada** temperatura; `2` cada dos temperaturas, y así sucesivamente. Las configuraciones son el archivo más grande (vea la sección 5).
- `OUTPUT_DIR` es el nombre de la carpeta de resultados.
- `GPUS = "all"` usa todas las GPU; `[0]` usa solo la primera.

4. Lo que verá mientras corre

- **La celda 2** informa el hardware encontrado, por ejemplo `using 2 GPU(s): [0, 1]`, o que corre en la CPU.
- **La celda 4** revisa sus parámetros e imprime cuántos sitios están activos y cuánto pesará el archivo de configuraciones. **No detiene** la corrida por el tamaño: asegúrese de tener espacio en disco.
- **La celda 5** corre la simulación e imprime una línea cada pocas temperaturas. Con varias GPU, cada línea empieza con la GPU que la imprimió, y los estados se reparten entre las GPU.
- **Las celdas 6 y 7** muestran una tabla resumen y cuatro gráficas: energía, |M|, Cv y χ contra la temperatura.

Como referencia, los valores por defecto (Heisenberg en una caja sc de 12 × 12 × 12, 32 réplicas, 19 temperaturas, 3000 barridos por temperatura) tardaron unos 20 segundos por estado en una T4 de Kaggle.

5. Los archivos de resultados

Todo se escribe en la carpeta `simulation_output`:

Archivo	Contenido	Forma
`magnetization.npy`	vector de magnetización media por sitio	[estado, réplica, temperatura, 3]
`configurations.npy`	cada espín de cada réplica	[estado, réplica, temperatura guardada, sitio, 3]
`sites.npy`	posición x, y, z de cada sitio activo	[sitio, 3]
`results.npz`	temperaturas, energía, $\langle E^2 \rangle$, $\langle \lvert M \rvert \rangle$, $\langle \lvert M \rvert^2 \rangle$, Cv, χ, nombres de los estados	vea abajo
`run.json`	todos los parámetros de la corrida, para repetirla	texto

La última dimensión `3` contiene las componentes x, y, z; Ising usa solo la primera y XY las dos primeras. Dentro de `results.npz`:

- `temperatures` — la lista de temperaturas;
- `energy`, `energy2`, `Mabs`, `M2` — $\langle E \rangle$, $\langle E^2 \rangle$, $\langle \lvert M \rvert \rangle$, $\langle \lvert M \rvert^2 \rangle$ por [estado, réplica, temperatura];
- `E_site`, `M_site`, `Cv`, `chi` — promediados sobre las réplicas, por [estado, temperatura];
- `state_names` — los nombres de los estados.

Cv y χ se calculan por sitio a partir de las fluctuaciones: $Cv = (\langle E^2 \rangle - \langle E \rangle^2) / (k_B T^2 N)$ y $\chi = N (\langle \lvert M \rvert^2 \rangle - \langle \lvert M \rvert \rangle^2) / (k_B T)$, donde N es el número de sitios activos.

Tamaño de `configurations.npy` en bytes: estados × réplicas × temperaturas guardadas × sitios × 12. En la corrida por defecto son 38 MB; una caja 3D grande con 128 réplicas y 200 temperaturas puede llegar a decenas de GB, así que en ese caso aumente `CONFIG_EVERY`.

Para abrir los archivos en Python:

```
import numpy as np
r = np.load("simulation_output/results.npz")
T, Cv = r["temperatures"], r["Cv"]                          # Cv[estado, temperatura]
M = np.load("simulation_output/magnetization.npy")          # M[estado, réplica,
↪  temperatura, componente]
C = np.load("simulation_output/configurations.npy", mmap_mode="r")   # se lee sin
↪  cargarlo entero
```

6. Ejemplos listos

Copie las líneas en la celda 1, reemplazando las que tienen el mismo nombre.

Modelo de Ising en 2D (transición exacta en T = 2.269):

```
MODEL = "ising"; CRYSTAL = "square"; BOX = [64, 64]; PERIODIC = [True, True]
STATES = [dict(name="Ising", J={1: 1.0})]
COORDS = ""; SHAPE = ""
T_MAX = 3.5; T_MIN = 1.5; T_STEP = 0.05
```

Esfera de Heisenberg de radio 8:

```
MODEL = "heisenberg"; CRYSTAL = "sc"; BOX = [17, 17, 17]; PERIODIC = [False, False,
↪  False]
COORDS = "spherical"; SHAPE = "r <= 8"
```

Nanodisco magnético con DMI, anisotropía y campo perpendicular (la aplicación del artículo):

```
MODEL = "heisenberg"; CRYSTAL = "sc"; BOX = [39, 39, 5]; PERIODIC = [False, False,
↪  False]
COORDS = "cylindrical"; SHAPE = "rho <= 18.3"
UNITS = "meV"
STATES = [dict(name="nanodisk", J={1: 1.0}, D=0.88, K1=0.177, Ks=0.139, K2=0.034,
↪  H=0.483, H_theta=0.0)]
T_MAX = 20.0; T_MIN = 0.1; T_STEP = 0.1
```

7. Cómo leer las gráficas

- **|M|** está cerca de 0 en caliente (espines desordenados) y crece en frío (espines ordenados).
- **Cv** y **χ** muestran un **pico** cerca de la temperatura de transición. En una caja finita el pico es redondeado y está algo desplazado; una caja más grande da un pico más agudo.
- Si un pico queda en la primera o en la última temperatura, la transición está fuera de su rango: amplíe `T_MAX`–`T_MIN`.
- Curvas con ruido: aumente `MEASUREMENT` o `REPLICAS`. Para ubicar una transición con más precisión, use un `T_STEP` más pequeño alrededor de ella.

8. Mensajes frecuentes y qué hacer

Mensaje	Qué significa	Qué hacer
`no usable GPU: running on the CPU`	no hay GPU, o su capacidad de cómputo es menor que 7.0	Kaggle: acelerador GPU T4 x2; Colab: GPU T4
`no admissible colouring for the periodic box …: using open boundaries`	ese tamaño de caja periódica no es compatible	nada (sigue con bordes abiertos), o cambie el tamaño de la caja
`D, K1, Ks and K2 need the Heisenberg model`	puso D o K en Ising o XY	use `MODEL = "heisenberg"` o quítelos
`unknown parameters ['Jx']`	un nombre de parámetro mal escrito	use solo J, D, K1, Ks, K2, H, H_theta, H_phi
`a shape needs its coordinate system`	`SHAPE` tiene algo y `COORDS` está vacío	llene `COORDS`
`unknown coordinate system`	`COORDS` está mal escrito	use cartesian, spherical o cylindrical
`the shape … contains no site of the box`	la figura está vacía o fuera de la caja	revise la fórmula y el tamaño de la caja
`crystal sc needs a box of 3 sizes`	la caja no corresponde al cristal	los cristales 2D llevan 2 tamaños y los 3D llevan 3
`need T_max >= T_min > 0 and T_step > 0`	temperaturas incorrectas	T_MAX mayor que T_MIN, ambas positivas
`state names must be distinct`	dos estados tienen el mismo nombre	cambie uno
`SyntaxError`	falta una coma, una comilla o un corchete en la celda 1	compare la línea con los ejemplos

9. Reproducibilidad

El cuadernillo escribe sus módulos a partir de texto guardado dentro de él y revisa cada uno con una huella digital (sha256), así que siempre corre el mismo código que el repositorio publicado `https://github.com/canamejoy/spin-colour-kernel`. Los mismos parámetros y la misma `SEED`, en el mismo tipo de hardware, dan los mismos resultados. `run.json` guarda todos los parámetros para que cualquiera pueda repetir la corrida.